



CS205 C/C++ Programming - Project 4: A Class to Describe a Matrix

Name: 殷海洲(Yin Haizhou)

SID: 12210226

Part 1 - Analysis

Requirement

1. The class should support different data types. It means that the matrix elements can be unsigned char, short, int, float, double, etc.
2. Do not use memory hard copy if a matrix object is assigned to another. Please carefully handle the memory management to avoid memory leaks and avoid releasing memory multiple times.
3. Operation overloading including but not limited to $=$, $==$, $+$, $*$, etc. Surely the matrix multiplication in Project 3 can be included.
4. Implement region of interest (ROI) to avoid memory hard copy. The region of a matrix Matrix a can be described by another object Matrix b. The two objects, a and b, share the same memory.

Requirement Analysis

Generally speaking, a versatile matrix class is required for certain purposes. It needs to store different types of data while it is also able to calculate or offer some extra functions.

The project starts with deep thinking about the practical situation of application for this matrix class so that the functions can be better designed.

First, ROI is one of most important part of the whole project despite it is listed in the last requirement. It needs to be clarified that it is ROI that makes project 4 meaningful since a function that tracks another matrix using one matrix is developed.

Besides, we may consider about interchangeability between different types of matrices, which enables us do calculation over the barrier of the datatype.

Moreover, it is a chance of implementing a complete program including using class, functions, overloading operators and etc.

Program Design

Then, to satisfy the requirements.

Initially, a class is defined as matrix. There are three vital member variables, columns, rows and the data of matrices. In order to avoid memory leak and hard copy, we use `shared_ptr` pointing to the data to intelligently manage dynamic memory.

And then, consider matrix types is different so we need to carefully manage its property and we introduce a enumeration `Datatype` in which there are `UNSIGNED_CHAR` , `SHORT` , `INT` , `FLOAT` , `DOUBLE` to indicate datatype of matrices. Using enumeration also provide advantages that we can easily manage them and make comparisons. This will be mentioned in subsequent context.

Additionally, consider applying ROI, we introduce a new member variable called `origin_cols` to reflect on offset in storage of data, since ROI matrices are specially managed while they share same class with normal matrices.

Because there are many situations we need to figure out the type of matrices and use this type to do something. A series of functions which will be elaborated later are implemented to provide necessary aids.

Then, we define constructor and destructor, functions to assign data for empty matrices. Meanwhile, there are other overloaded operators for calculation. The example will be demonstrated in Part 3.

Part 2 - Code

2.0 Preprocess

```
#include <iostream>
#include <memory>
#include <cstring>

using namespace std;
```

```

union entry
{
    unsigned char uc;
    short sh;
    int in;
    float fl;
    double db;
};

//we use union as entries so that we make sure that data is aligned

typedef enum Datatype{
    UNSIGNED_CHAR,SHORT,INT,FLOAT,DOUBLE
}Datatype;

//using enumeration to indicate the type

```

2.1 Class member variable

```

class Matrix
{
    private:
        size_t rows;
        size_t cols;
        size_t origin_cols;
        shared_ptr<entry[]> data;
    public:
        Datatype datatype;

```

2.2 Constructor & Destructor

```

Matrix(): rows(0), cols(0),origin_cols(0),data(nullptr),datatype(DOUBLE){}
Matrix(size_t i): rows(1), cols(i), origin_cols(i),data(new entry[i]),
datatype(DOUBLE){}
Matrix(size_t r, size_t c): rows(r), cols(c), origin_cols(c),
data(new entry[r*c]),datatype(DOUBLE){}

```

2.2.1 Copy constructor

```
Matrix(const Matrix &m):rows(m.rows),cols(m.cols),
origin_cols(m.origin_cols),data(m.data),datatype(m.datatype){}
//copy constructor
```

2.2.2 Copy assignment

```
Matrix& operator=(const Matrix &m)
{
    if (this != &m) {
        this->rows = m.rows;
        this->cols = m.cols;
        this->origin_cols = m.origin_cols;
        this->datatype = m.datatype;
        this->data = m.data;
    }
    return *this;
}
```

2.2.3 Destructor

```
~Matrix(){}

```

2.3 Member functions & Overloaded operators

2.3.1 Data assignment

Since Matrices are built empty, we need some ways to fill them with numbers or characters.

```
template <typename T>
void Matrix::genedata(Matrix &m, T*arr,size_t n)
{
    if (n == 0)
    {
        cerr << "No data is in." << endl;
    }
}
```

```

        return;
    }
    if (m.rows == 0 || m.cols == 0 || m.rows*m.cols < n)
    {
        cerr << "Size of your matrix is too small." << endl;
        return;
    }
    m.datatype = Whatisit(arr,n);
    switch(m.datatype)
    {
        case DOUBLE:
            for(size_t a = 0;a<m.rows*m.cols;a++)
                m.data.get()[a].db = arr[a];
            break;
        case FLOAT:
            for(size_t a = 0;a<m.rows*m.cols;a++)
                m.data.get()[a].fl = arr[a];
            break;
        case INT:
            for(size_t a = 0;a<m.rows*m.cols;a++)
                m.data.get()[a].in = arr[a];
            break;
        case SHORT:
            for(size_t a = 0;a<m.rows*m.cols;a++)
                m.data.get()[a].sh = arr[a];
            break;
        case UNSIGNED_CHAR:
            for(size_t a = 0;a<m.rows*m.cols;a++)
                m.data.get()[a].uc = arr[a];
            break;
        default:
            throw invalid_argument("No appropriate datatype.");
    }
}

```

2.3.2 Data reader

```

template <typename V>
V Matrix::getelement(const Matrix &m, size_t r, size_t c)
{
    V element;
    if (r >= m.rows || c >= m.cols)

```

```

{
    throw invalid_argument("Out of range.");
}
size_t loc = r*m.origin_cols+c;
switch(m.datatype)
{
    case DOUBLE:
        element = m.data.get()[loc].db;
        break;
    case FLOAT:
        element = m.data.get()[loc].fl;
        break;
    case INT:
        element = m.data.get()[loc].in;
        break;
    case SHORT:
        element = m.data.get()[loc].sh;
        break;
    case UNSIGNED_CHAR:
        element = m.data.get()[loc].uc;
        break;
    default:
        throw invalid_argument("No appropriate datatype.");
}
return element;
}

```

2.3.3 Arithmetic operations

Addition

```

Matrix Matrix::operator+(const Matrix &m)
{
    if(this->rows != m.rows || this->cols != m.cols)
    {
        cerr << "Different size." << endl;
        return *this;
    }
}

```

```

    if(this->rows == 0 || this->cols == 0)
    {
        cerr << "Empty matrix." << endl;
        return *this;
    }
    Matrix temp(m.rows,m.cols);
    if (this->datatype > m.datatype)
        temp.datatype = this->datatype;
    else
        temp.datatype = m.datatype;
    for(size_t a = 0; a < m.rows; a++)
    {
        for(size_t b = 0; b < m.cols; b++)
        {
            switch(this->datatype)
            {
                case DOUBLE:
                    this->data.get()[a*this->origin_cols+b].db += getelement<double>(m, (a*m.rows+b) /
                    break;
                case FLOAT:
                    this->data.get()[a*this->origin_cols+b].fl += getelement<float>(m, (a*m.rows+b) /
                    break;
                case INT:
                    this->data.get()[a*this->origin_cols+b].in += getelement<int>(m, (a*m.rows+b) / m.
                    break;
                case SHORT:
                    this->data.get()[a*this->origin_cols+b].sh += getelement<short>(m, (a*m.rows+b) /
                    break;
                case UNSIGNED_CHAR:
                    this->data.get()[a*this->origin_cols+b].uc += getelement<unsigned char>(m, (a*m.ro
                    break;
                default:
                    throw invalid_argument("Unable to add.");
            }
        }
    }
    return temp;
}

Matrix Matrix::operator+=(const Matrix &m)

```

```

{
    if(this->rows != m.rows || this->cols != m.cols)
    {
        cerr << "Different size." << endl;
        return *this;
    }
    if(this->rows == 0 || this->cols == 0)
    {
        cerr << "Empty matrix." << endl;
        return *this;
    }
    if (this->datatype > m.datatype)
        this->datatype = this->datatype;
    else
        this->datatype = m.datatype;
    for(size_t a = 0; a < m.rows; a++)
    {
        for(size_t b = 0; b < m.cols; b++)
        {
            switch(this->datatype)
            {
                case DOUBLE:
                    this->data.get()[a*this->origin_cols+b].db += getelement<double>(m, (a*m.rows+b) /
                    break;
                case FLOAT:
                    this->data.get()[a*this->origin_cols+b].fl += getelement<float>(m, (a*m.rows+b) /
                    break;
                case INT:
                    this->data.get()[a*this->origin_cols+b].in += getelement<int>(m, (a*m.rows+b) / m.
                    break;
                case SHORT:
                    this->data.get()[a*this->origin_cols+b].sh += getelement<short>(m, (a*m.rows+b) /
                    break;
                case UNSIGNED_CHAR:
                    this->data.get()[a*this->origin_cols+b].uc += getelement<unsigned char>(m, (a*m.ro
                    break;
                default:
                    throw invalid_argument("Unable to add.");
            }
        }
    }
    return *this;
}

```



```
}
```

Subtraction

```
Matrix Matrix::operator-(const Matrix &m)
{
    if(this->rows != m.rows || this->cols != m.cols)
    {
        cerr << "Different size." << endl;
        return *this;
    }
    if(this->rows == 0 || this->cols == 0)
    {
        cerr << "Empty matrix." << endl;
        return *this;
    }
    Matrix temp(m.rows,m.cols);
    if (this->datatype > m.datatype)
        temp.datatype = this->datatype;
    else
        temp.datatype = m.datatype;
    for(size_t a = 0; a < m.rows * m.cols; a++)
    {
        switch(temp.datatype)
        {
            case DOUBLE:
                temp.data.get()[a].db = getelement<double>(*this, a / m.cols, a % m.cols) - getelement<double>(m.data, a / m.cols, a % m.cols);
                break;
            case FLOAT:
                temp.data.get()[a].fl = getelement<float>(*this, a / m.cols, a % m.cols) - getelement<float>(m.data, a / m.cols, a % m.cols);
                break;
            case INT:
                temp.data.get()[a].in = getelement<int>(*this, a / m.cols, a % m.cols) - getelement<int>(m.data, a / m.cols, a % m.cols);
                break;
            case SHORT:
                temp.data.get()[a].sh = getelement<short>(*this, a / m.cols, a % m.cols) - getelement<short>(m.data, a / m.cols, a % m.cols);
                break;
        }
    }
    return temp;
}
```

```

        break;
    case UNSIGNED_CHAR:
        temp.data.get()[a].uc = getelement<unsigned char>(*this, a / m.cols, a % m.cols) - get
        break;
    default:
        throw invalid_argument("Unable to subtract.");
    }
}
return temp;
}

```

Multiplication

```

Matrix Matrix::operator*(const Matrix &m)
{
    if(this->cols != m.rows)
    {
        cerr << "Different size." << endl;
        return *this;
    }
    if(this->rows == 0 || this->cols == 0 || m.rows == 0 || m.cols == 0)
    {
        cerr << "Empty matrix." << endl;
        return *this;
    }
    Matrix temp(this->rows, m.cols);
    if (this->datatype > m.datatype)
        temp.datatype = this->datatype;
    else
        temp.datatype = m.datatype;
    for(size_t a = 0; a < this->rows; a++)
    {
        for(size_t b = 0; b < m.cols; b++)
        {
            size_t loc = a*temp.origin_cols+b;
            switch(temp.datatype)
            {

```

```

        case DOUBLE:
            temp.data.get()[loc].db = 0;
            for(size_t c = 0; c < this->cols; c++)
                temp.data.get()[loc].db += getelement<double>(*this, a, c) * getelement<double>(
            break;
        case FLOAT:
            temp.data.get()[loc].fl = 0;
            for(size_t c = 0; c < this->cols; c++)
                temp.data.get()[loc].fl += getelement<float>(*this, a, c) * getelement<float>(
            break;
        case INT:
            temp.data.get()[loc].in = 0;
            for(size_t c = 0; c < this->cols; c++)
                temp.data.get()[loc].in += getelement<int>(*this, a, c) * getelement<int>(m, c
            break;
        case SHORT:
            temp.data.get()[loc].sh = 0;
            for(size_t c = 0; c < this->cols; c++)
                temp.data.get()[loc].sh += getelement<short>(*this, a, c) * getelement<short>(
            break;
        case UNSIGNED_CHAR:
            temp.data.get()[loc].uc = 0;
            for(size_t c = 0; c < this->cols; c++)
                temp.data.get()[loc].sh += getelement<unsigned char>(*this, a, c) * getelement
            break;
        default:
            throw invalid_argument("Unable to multiply.");
    }
}
}
return temp;
}

```

output operation

```

ostream & operator<<(ostream &os, const Matrix &m)
{
    if (m.rows == 0 || m.cols == 0)
    {
        cerr << "Empty matrix." << endl;
    }
}

```

```

        return os;
    }
    for(size_t a = 0;a<m.rows;a++)
    {
        for(size_t b = 0;b<m.cols;b++)
        {
            switch(m.datatype)
            {
                case DOUBLE:
                    os << m.data.get()[a*m.origin_cols+b].db << " ";
                    break;
                case FLOAT:
                    os << m.data.get()[a*m.origin_cols+b].fl << " ";
                    break;
                case INT:
                    os << m.data.get()[a*m.origin_cols+b].in << " ";
                    break;
                case SHORT:
                    os << m.data.get()[a*m.origin_cols+b].sh << " ";
                    break;
                case UNSIGNED_CHAR:
                    os << m.data.get()[a*m.origin_cols+b].uc << " ";
                    break;
                default:
                    throw invalid_argument("No data.");
            }
        }
        os << endl;
    }
    return os;
}

```

2.4 Global function

2.4.1 Datatype

Datatype functions help determine the datatype and return as an element of enumeration, which helps show priority of datatype and manipulate matrices. We

make it clear that matrices are of what datatype but we need figure out how to use this crucial information.

My answer will be we could convert the `type_info` using `typeid()` to Datatype.

```
Datatype WhoamI(const type_info& t)
{
    if (t == typeid(double))
        return DOUBLE;
    else if (t == typeid(float))
        return FLOAT;
    else if (t == typeid(int))
        return INT;
    else if (t == typeid(short))
        return SHORT;
    else if (t == typeid(unsigned char))
        return UNSIGNED_CHAR;
    else
        throw invalid_argument("Unknown type");
}

template <typename U>
Datatype Whatisit(U* arr, size_t n)
{
    Datatype temp = UNSIGNED_CHAR;
    for (size_t a = 0; a < n-1; a++)
    {
        const type_info& t1 = typeid(arr[a]);
        const type_info& t2 = typeid(arr[a + 1]);
        Datatype d1 = WhoamI(t1);
        Datatype d2 = WhoamI(t2);
        if (d1 > d2)
            temp = d1;
        else
            temp = d2;
    }
    return temp;
}
```

2.4.2 Type information

It is composed when I writing the functions above, I thought it would be useful but turned to be a source of bugs. (Abandoned)

```
const type_info & Mytype(Datatype d) {  
    switch (d) {  
        case DOUBLE:  
            return typeid(double);  
        break;  
        case FLOAT:  
            return typeid(float);  
        break;  
        case INT:  
            return typeid(int);  
        break;  
        case SHORT:  
            return typeid(short);  
        break;  
        case UNSIGNED_CHAR:  
            return typeid(unsigned char);  
        break;  
        default:  
            throw invalid_argument("Unknown Datatype");  
    }  
}
```

2.5 ROI

ROI matrix is a matrix but shared memory with another matrix. The problem is to locate the data so we can do some operation to change the data of the target matrix and vice versa.

```
Matrix Matrix::createROI(size_t r, size_t c, size_t h, size_t w)  
//row location,column location,height,width  
{
```

```

Matrix temp(h,w);
if (r+h > this->rows || c+w > this->cols)
{
    cerr << "Out of range." << endl;
    return temp;
}
temp.origin_cols = this->origin_cols;
temp.datatype = this->datatype;
size_t offset = r * this->origin_cols + c;
temp.data = shared_ptr<entry[]>(this->data.get() + offset, [](entry*){});
return temp;
}

```

Part 3 - Result & Verification

3.1 Verifying Code

create a 2x2 matrix m1

create a 3x3 matrix m2

create an identity 3x3 matrix m3

m4 is the product of m2 and m3

m5 inspect m4 as ROI

If we change contents in m4, the one in m5 will vary as well, indicating that they share same memory.

```

#include <iostream>
#include <memory>
#include "class.hpp"

using namespace std;

int main()
{
    Matrix m1(2, 2);
    Matrix m2(3, 3);
    Matrix m3(3, 3);
    int arr1[] = {2,2,2,2};
    double arr2[] = {1,2,3,4,5,6,7,8,9};
}

```

```

float arr3[] = {1,0,0,0,1,0,0,1};
m1.genedata(m1, arr1,sizeof(arr1)/sizeof(int));
m2.genedata(m2, arr2,sizeof(arr2)/sizeof(double));
m3.genedata(m3, arr3,sizeof(arr3)/sizeof(float));

Matrix m4 = m2 * m3;
Matrix m5 = m4.createROI(0, 0, 2, 2);
//ROI test, shared memory
cout << "m4 = "<< endl;
cout << m4 << endl;
cout << "m5 = "<< endl;
cout << m5<< endl;
m4 += m3;
cout << "m4 = "<< endl;
cout << m4 << endl;
cout << "m5 = "<< endl;
cout << m5<< endl;

m5 += m1;
cout << "m5 = "<< endl;
cout << m5<< endl;

Matrix m6;
m6 = m3;//copy assignment test
cout << "m6 = "<< endl;
cout << m6<< endl;

unsigned char arr4[9] = {'a','b','c','d','e','f','g','h'};
Matrix m7(3, 3);
m7.genedata(m7, arr4,sizeof(arr4)/sizeof(unsigned char));
Matrix m8 = m7 * m6;
cout << "m7 = "<< endl;
cout << m7<< endl;
cout << "m8 = "<< endl;
cout << m8<< endl;//char matrix multiplication test

Matrix m9(3, 3);
short arr5[] = {1,2,3,4,5,6,7,8,9};
m9.genedata(m9, arr5,sizeof(arr5)/sizeof(short));
Matrix m10 = m9 - m4;
cout << "m10 = "<< endl;
cout << m10<< endl;//short matrix subtraction test
}

```


Result

ROI test

The result is supposed to show that ROI matrix tracking the target matrix.

```
m4 =  
1 2 3  
4 5 6  
7 8 9
```

```
m5 =  
1 2  
4 5
```

```
m4 =  
2 2 3  
4 6 6  
7 8 10
```

If we add m5 with a 2x2 matrices with elements 2

```
m5 =  
2 2  
4 6
```

```
m5 =  
4 4  
6 8
```

Other basic arithmetic test

```
m6 =  
1 0 0  
0 1 0  
0 0 1  
  
m7 =  
a b c  
d e f  
g h  
  
m8 =  
97 98 99  
100 101 102  
103 104 0
```

m6 is a result of using copy assignment

PS: decimal "a" equals 97

```
m10 =  
-3 -2 0  
-2 -3 0  
0 0 -1
```

Part 4 - Difficulty & Solution

4.1 Transfer between datatypes

It is common for most of students to consider using a class template to handle different datatype as following.

```
template <typename T>  
class Matrix  
{  
    private:  
        size_t rows;  
        size_t cols;  
        shared_ptr<T> data;  
    public:  
        Matrix() : rows(0), cols(0), data(new T[rows * cols]){}  
        Matrix(size_t i) : rows(1), cols(i), data(new T[rows * cols]){}  
        Matrix(size_t r, size_t c) : rows(r), cols(c), data(new T[rows * cols]){}  
        Matrix(const Matrix & m) : rows(m.rows), cols(m.cols), data(m.data){}  
        ~Matrix() {}  
        Matrix & operator=(const Matrix & m){  
            return *this;  
        }  
        \\.....  
}
```

However, the very big problem is that the template class is independent for each different datatype, so it was soon discarded even if a sufficiently complete work had been derived.

Then we would again consider how to load different datatype.

A solution will be using a union to replace a certain datatype. There are two advantages. First is that the data are now aligned so it will be easy for user to manipulate (read and write), and the other is that you can load all kinds of data together in the same 8 bytes.

```
union entry
{
    unsigned char uc;
    short sh;
    int in;
    float fl;
    double db;
};
```

But it also incurs problems of reading, which means users need specialized functions to use the data. As mentioned above many functions to check the datatype are used for this reasons.

4.2 Simplify the project

When the project get bigger, it will become gradually harder to manage each function, so it is necessary to divide a big problem into parts.

For instance, calculation requires a bunch of operations and it is annoying to take care of trivial things every single try, so we could use functions directly to get what we need. In this example a process to extract numbers in unions to avoid complexity and improve efficiency in maintenance of codes.

```
template <typename V>
V Matrix::getelement(const Matrix &m, size_t r, size_t c)
{
    V element;
    if (r >= m.rows || c >= m.cols)
    {
        throw invalid_argument("Out of range.");
    }
}
```

```

size_t loc = r*m.origin_cols+c;
switch(m.datatype)
{
    case DOUBLE:
        element = m.data.get()[loc].db;
        break;
    case FLOAT:
        element = m.data.get()[loc].fl;
        break;
    case INT:
        element = m.data.get()[loc].in;
        break;
    case SHORT:
        element = m.data.get()[loc].sh;
        break;
    case UNSIGNED_CHAR:
        element = m.data.get()[loc].uc;
        break;
    default:
        throw invalid_argument("No appropriate datatype.");
}
return element;
}

```

4.3 Dynamic memory management

Although smart pointers are used, it doesn't mean OK, now you can ignore the problem of dynamic memory. I encountered still twice the double free problem during debugging, and I spent 1 hour more to solve it caused for my arrogance. It is finally turned out to be two smart pointers pointed to the same memory and none of them knew each other.

Solution: I reconstructed my program to avoid this problem.

Part 5 Appendix

experience, comments and reference

5.1 Experience and Comments

In conclusion, just like our professor mentioned in his class, there are many things you should take care of or you code for little time but debug for three weeks(a bit exaggeration). There are also many details that programmers have to examine and carefully organize so that your codes would not run unexpectedly or silently seed invisible bugs.

Although matrices have been our topics over three weeks and might still appear in the next one, we still remain far and far from a really complete program commodity and experient in matrices which implies that we have a great large space to start our rat race(内卷) . Dramatically these days students write much more pages for their project reports ~~Not because good examples are showed but students realize the essence and joy of coding.~~

Hopefully, my professor could somehow enjoy my report and works.

5.2 Reference

CSDN Blog [【智能指针】shared_ptr基本用法和原理（共享指针）](#)

CSDN Blog [计算机视觉-图像的ROI区域](#)